



1. System Overview

This document defines a **high-performance, scalable, and cloud-native backend architecture** for a mobile application. The system is designed to support:

- High concurrency
- Low latency content delivery
- Efficient data retrieval
- Scalable media storage
- Distributed caching

The architecture follows a **modular, stateless, and horizontally scalable design**.

2. High-Level Architecture

Client (Flutter Application)

↓

Internet (HTTPS)

↓

CDN Layer (CloudFront)

↓

Backend Hosting (Railway)

↓

Application Server (Node.js + Express.js)

↓

| | | |

Database Layer Cache Layer Storage Layer |

(Neon PostgreSQL) (Redis) (AWS S3) |

3. Technology Stack

3.1 Client Layer

- Framework: Flutter
- Language: Dart
- Networking: HTTP/HTTPS APIs
- Local Caching: In-memory + disk caching

3.2 Backend Layer

- Runtime: Node.js
- Framework: Express.js
- Architecture: RESTful APIs
- Environment: Railway (cloud PaaS)

3.3 Database Layer

- System: PostgreSQL
- Provider: Neon (Serverless)
- Features:
 - JSONB support
 - Indexing (B-Tree, GIN)
 - Connection pooling

3.4 Storage Layer

- Service: Amazon S3
- Type: Object Storage
- Data Type: Binary objects (images, videos, files)

3.5 CDN Layer

- Service: Amazon CloudFront
 - Purpose: Global content delivery network
 - Integration: Connected to S3 as origin
-

3.6 Caching Layer

- Service: Redis
 - Type: In-memory data store
 - Usage:
 - API response caching
 - Session/token storage
 - Frequently accessed data
-

4. Detailed Component Architecture

4.1 Client Layer

Responsibilities:

- Sends API requests over HTTPS
 - Handles UI rendering
 - Implements:
 - Lazy loading
 - Pagination
 - Image caching
 - Displays media via CDN URLs
-

4.2 CDN Layer (CloudFront)

Responsibilities:

- Serves static content (images/videos) from edge locations
- Reduces latency by delivering content from nearest server

- Caches frequently accessed media

Flow:

User → CloudFront → (Cache Hit → Serve)

→ (Cache Miss → Fetch from S3 → Cache → Serve)

Benefits:

- Reduced load on backend
 - Faster media delivery
 - Global scalability
-

4.3 Backend Layer (Node.js + Express)

Responsibilities:

- API routing
- Business logic execution
- Authentication & authorization
- Communication with DB, Redis, and S3

Key Characteristics:

- Stateless design
 - Horizontal scalability
 - JSON-based communication
-

4.4 Database Layer (Neon PostgreSQL)

Responsibilities:

- Persistent data storage
- Relational + semi-structured data handling

Design Principles:

- Normalization (up to 3NF)
- JSONB for flexible schema
- Indexing for performance

Index Types:

- B-Tree (default)
 - GIN (for JSONB queries)
-

4.5 Cache Layer (Redis)

Responsibilities:

- Store frequently accessed data in memory
- Reduce database load
- Improve response time

Use Cases:

- API response caching
- Session/token caching
- Rate limiting
- Feed caching

Example Flow:

Request → Check Redis

→ If HIT → Return data

→ If MISS → Query DB → Store in Redis → Return

TTL Strategy:

- Short-lived cache for dynamic data
 - Long-lived cache for static data
-

4.6 Storage Layer (S3)

Responsibilities:

- Store binary files
- Provide object URLs
- Ensure durability and availability

Best Practices:

- Unique file naming (UUID)
- Store only URLs in database
- Use pre-signed URLs for secure uploads

5. Data Flow Architecture

5.1 API Request Flow

Client → Backend → Redis Check

→ (Hit → Return)

→ (Miss → DB Query → Cache → Return)

5.2 Media Upload Flow

Client → Backend → Upload to S3 → Generate URL → Store in DB → Return URL

5.3 Media Delivery Flow

Client → CloudFront → (Cache Hit → Serve)

→ (Cache Miss → Fetch S3 → Cache → Serve)

6. API Design Specifications

- Protocol: HTTPS
- Format: JSON
- Architecture: RESTful

Standard Response:

```
{  
  "success": true,  
  "data": {},  
  "error": null  
}
```

7. Performance Optimization

7.1 Backend

- Redis caching
- Query optimization
- Connection pooling

7.2 Database

- Indexing (B-Tree, GIN)
- Pagination
- Avoid full table scans

7.3 Media

- Image compression before upload
- CDN delivery via CloudFront

7.4 Client

- Lazy loading
- Infinite scroll
- Cached image loading

8. Security Architecture

- HTTPS encryption
- Environment variables for secrets
- JWT-based authentication (ready)
- Input validation
- Role-based access control (RBAC)

S3 Security:

- Pre-signed URLs
 - Bucket policies
-

9. Scalability Strategy

- Stateless backend → horizontal scaling
 - Redis reduces DB load
 - Neon auto-scales database
 - S3 + CloudFront handle media scaling
-

10. Fault Tolerance

- Retry mechanisms for failed requests
 - Graceful error handling
 - Logging and monitoring
-

11. Logging & Monitoring

- Application logs via Railway
 - Error tracking
 - Performance metrics
 - Future integration:
 - Prometheus
 - Grafana
-

12. Deployment Strategy

- GitHub-based CI/CD
 - Auto deployment on push
 - Environment separation:
 - Development
 - Production
-

13. Future Enhancements

- Microservices architecture

- Load balancer
 - Multi-region deployment
 - Advanced caching strategies
-

14. Conclusion

This system is designed to deliver:

- ⚡ Low latency (via CDN + Redis)
- 📈 High scalability (serverless + cloud-native)
- 🔒 Strong security
- 🧠 Efficient data handling

The integration of **CloudFront (CDN)** and **Redis (caching)** from the initial stage ensures production-grade performance and user experience.